

II. Additional AI and Machine Learning Technologies

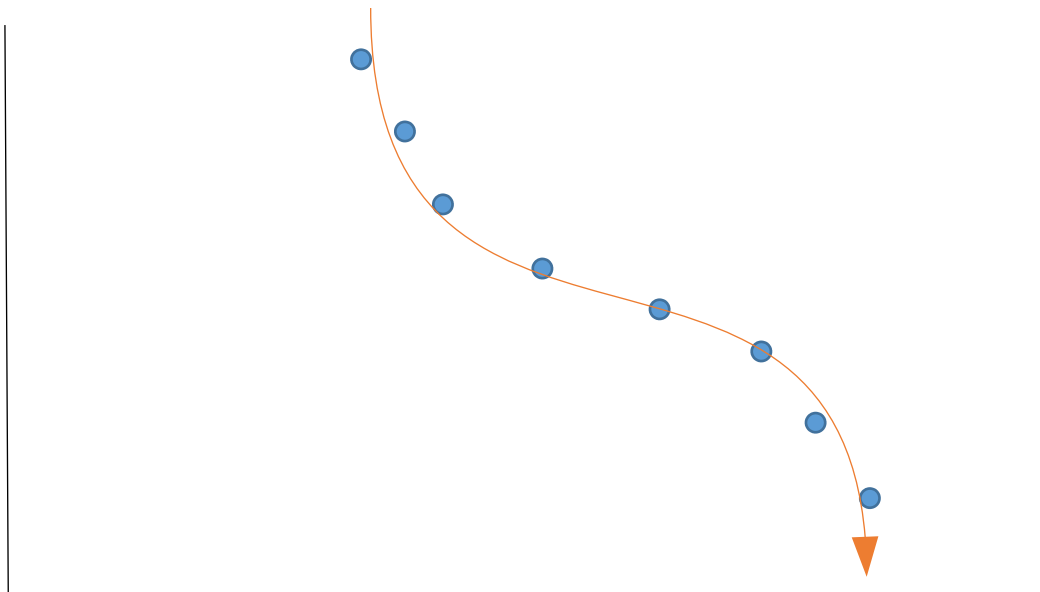
Neural Networks and Deep Learning

The most significant increases in Machine Learning performance over the past decade have come from a technique known as *neural networks*. A neural network is a supervised learning model that learns how to generate desired outputs (labels) from a set of inputs. Like all supervised Machine Learning, this process starts by training the neural network using a labeled data set of inputs and desired outputs.¹

A neural network can have one or more inputs and one or more outputs. The inputs and outputs of a neural network are always numbers although most often those numbers are merely the medium by which relevant real-world concepts are represented. When our objective requires that inputs or outputs be something besides numbers, say the text of a hotel review and a class representing whether that review is positive or negative, we nevertheless must use numbers to represent these concepts. It will help to simply accept this fact initially and leave for a later discussion how exactly we translate things like images or text into numbers.

The training process gives the neural network an internal representation of how to generate each output number from the input numbers. We can use an X-Y graph in the Cartesian plane to envision this representation for the simplest situation in which there is just one input number and one output number that is produced from that input number.

¹ Although a neural network always needs training data in the form of inputs and labels (desired outputs), sometimes the training data appears to have no labels at all. For example, in word vector training we will see that we simply give the neural network large amounts of text, e.g., millions of Wikipedia pages, but that text has no class labels or other human-provided labels. In these cases the input text also contains the desired outputs for the particular neural network training objective under a self-supervised learning paradigm; it might not be obvious to the observer exactly which parts of the unlabeled data are being used for the labels.



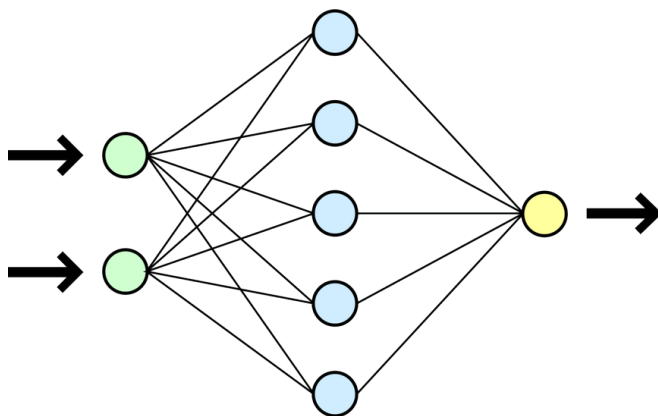
This figure includes eight dots in the X-Y plane. Each dot has an X coordinate representing the input value and a Y coordinate representing the output value. These dots represent the training data: eight inputs X values and the corresponding desired outputs (labels) which is a Y value. Through the training process, the neural network learns to build an internal representation of how to generate an output (a Y coordinate) from any arbitrary input (an X coordinate). The neural network's internal representation is illustrated here as a curve. Given any arbitrary X coordinate, this curve determines one corresponding Y coordinate. The neural network generates that Y output when provided with that X input.

The curve in the figure above is extremely simple. A more complicated curve could have many sudden twists or turns with dramatic increases and decreases in slope as one proceeds across the curve. The simplest curve is a line, representing a very simple (linear) relationship between input numbers and output numbers. In the example above only a single number – an X value – is used as an input and consequently the output Y value is produced using only this one number. A more complicated neural network might use several numbers as an input. For example, a neural network that predicts a person's credit score might utilize dozens of numbers to make its prediction, such as the customer's annual income, current loan balance, history of payments, etc. Intuitively, the more inputs (numbers) that are available for a particular decision that the neural network must make, the

more information is conveyed to the neural network, and we hope that the ensuing prediction by the network is more accurate.

A neural network is composed of several *units*. There are *input units* that receive the numbers used as inputs to the neural network, *output units* that produce the final output numbers from the given inputs, and *hidden units* that are involved in intermediate processing to transform input numbers into output numbers. Each unit performs relatively simple mathematical operations on one or more numbers it receives. The unit generates another number – the unit’s output - from the numbers it receives.

The figure below illustrates a neural network with two input units on the left, one output unit on the right, and a single layer of five hidden units between the two. Neural network diagrams are typically arranged so information flows from left to right, or from bottom to top, as we transition from inputs received to outputs generated.



Units are connected to each other by lines in the figure above. The lines represent where a unit receives the numbers that it processes. The units in the middle (hidden) layer receive numbers from the input units, but not from each other. The output unit receives numbers from only the units in the hidden layer, but not from any of the input units.

This figure has a single hidden layer and just a few units. Most neural networks include several hidden layers. The number of layers very roughly corresponds to the processing power of the neural network and the accuracy of decisions the neural network can make. That is, with more layers the neural network generally can “understand” and represent more complex relationships between the

inputs and the outputs. The added power conferred by adding layers is not without cost. Typically, each layer significantly increases the training time of the neural network.

More layers are typically necessary when we use a neural network to process complicated data, such as natural language or images, or to render complex decisions, such as determining whether an MRI of a patient's lungs reveals a carcinoma. Adding units to a layer can also provide the neural network with the additional processing power to represent more complicated relationships.

However, adding layers to a neural network generally provides much more power than adding the same number of units to existing layers. A network with many layers is considered “deep” and the term *deep learning* generally refers to using a neural network with a large number of layers.

Therefore deep learning generally implies a network with greater power and thus a greater ability to model complex phenomena.

Designing neural networks for classification

Neural networks can be trained to perform a wide variety of tasks. Some tasks require regression, i.e., that the neural network outputs one or more numbers for each input. For example, an input to a neural network can be a description of a house for sale, and the desired output is the predicted sales price of the house. Regression very comfortably matches the capabilities of neural networks because the outputs of neural networks are always numbers.

However, other tasks require classification – i.e., identifying which of a set of categories an input belongs to. For example, a neural network operating on an assembly line may need to decide whether an image of each component being assembled appears to be defective or not defective. One or more numbers does not seem to match such categorical outputs. However the standard manner of adapting neural network to classification is to consider the networks outputs to be akin to probabilities of the input being assigned to a particular class. The class that the input is assigned to would simply be the class with the highest probability.

When there are two classes, known as *binary classification*, a single number can be used to represent the probabilities for both classes. That single number directly represents the probability that one of the classes, the primary class, is correct. Since the probabilities for all classes must sum to 1.0 (representing 100%), the probability of the remaining class is also completely determined by

that number. For example, if the network produces an output of 0.25 this means that the primary class is 25% likely and the remaining class is 75% likely. We are free to arbitrarily designate which of the two classes is the primary class; this does not affect the network's performance.

When there are three or more classes there are various approaches to how to use numbers to represent the classes. One common way is for the network to output a number for each class. It is simple to apply a mathematical transformation so that each number is non-negative and all numbers sum to 1.0. Thus the network outputs a probability for each class.

During the process of training a neural network, when the network produces an incorrect classification this means the network has produced a probability for the correct class that is too low, and the probabilities of one or more incorrect classes are unduly high. Successful training would then increase the probability of the correct class and reduce the probability of the incorrect classes.

There can be additional benefits of interpreting the outputs of a neural network classifier as probabilities. One benefit lies in monitoring the progress of classifier training. There can be situations where the network does not produce high-quality output for a particular input even after significant training. The network may incorrectly classify that input, which means an incorrect class has the highest probability. A situation only slightly better is where the network correctly classifies that input, but just barely. For example, in binary classification the correct class may be assigned a probability of 50.1% while the incorrect class is assigned a probability of 49.9%. In this situation where the correct class has the highest probability but another class has a probability almost as high, one can interpret the classifier as having low confidence in its decision.

Although the network assigns the highest probability to the correct class, the situation is clearly precarious. The slightest change in training or retraining the network could reduce the probability of the correct class to below 50%. Also, a different input that is very similar to the problematic input might be assigned a probability less than 50.1%, perhaps low enough to be incorrectly classified.

Analyzing incorrect classifications or correct classifications with low confidence can reveal some things about:

- (a) the data used in training

It is possible that the training data contains incorrect or contradictory labels. This in turn forces the network to learn something that might be incorrect, or which may be difficult or impossible to learn.

- (b) the network

It is possible that the neural network as currently designed lacks the power to make sufficiently accurate assessments about the data. For example, the network may not have a sufficient number of layers or units, or perhaps the nature of the inputs or outputs require special network structure to process efficiently. For example, to better process image or speech inputs it often helps to add specialized components to the neural network.

- (c) the nature of the input that is incorrectly classified

The incorrectly classified input may be an outlier or otherwise anomalous. Although that input belongs to one class, it might possess unusual characteristics. These unusual characteristics might typically be possessed by inputs from another class, or rarely possessed by inputs from the correct class.

Some Caveats

A word of caution about neural network capabilities is warranted. Some statements about machine learning and in particular neural networks are true, but extremely misleading without the proper context and considerations. Understanding the subtle limits of neural networks will allow you to more critically review neural network tools and the claims that are often made about neural network performance.

For example, a very popular, and generally true, assertion is:

“a neural network can learn to perform any task once it is trained using a sufficient amount of training data”

In other words, any relationship between the inputs and the output, no matter how complex, can be learned given a sufficient number of labeled examples. Therefore we can in theory have a neural network perform any of a wide variety of tasks if we have the right kind of training data. Further

supporting this assertion is the fact that researchers have mathematically proven that a neural network can in principle approximate any function.²

This statement is only true given certain assumptions that are often unaddressed. First, even though it is possible for a neural network *to be trained* to perform the desired way, it is not necessarily true that *the particular training process* being used to train the network will successfully lead to optimal training. In fact, it is possible that no known learning algorithm will suffice.

A more significant assumption involves the training data set. First, it assumes something about *the nature of the data set*: that the inputs actually contain the information that is required to derive the output. For example, imagine we tried to learn how to predict the GPA of a student given the student's height and shoe size. Even if we had an exhaustive training data set that accurately represented the inputs (students' heights and shoe sizes) and outputs (students' GPA) of all students in the entire US over the last 50 years, the desired output (GPA) likely cannot be derived with acceptable accuracy from the specified inputs (height and shoe size); the inputs and outputs are almost entirely unrelated. The neural network cannot learn any useful relationship between inputs and outputs, so it could not learn to make the desired prediction with any high accuracy.

It is sometimes easy for a human to inspect a data set and know whether the inputs contain enough information to calculate the outputs, or whether the inputs are unrelated to the desired outputs. However in many circumstances it will not be clear whether the available inputs have all of the information required to accurately generate the desired outputs. It is possible that only some form of correlation exists between the inputs and outputs, but necessary additional information is lacking. The strength of this correlation will determine the accuracy the neural network could attain in its outputs, and the nature of the errors it makes.

Furthermore, the input-output pairs in the training data must cover the diverse range of possible inputs and outputs. For example, imagine we trained a system to accurately determine whether an image of a component on an assembly line is defective. In our training data the inputs could be the component images from the assembly line over the last 40 years, and the outputs would be the true nature of the components: defective or not. Our training data is appropriate for learning this type of input, but not for inputs with very different forms, such as images of slightly different components

² Neural networks are often called a "universal approximator". The first universal approximation theorem was proven in 1989 by Thayer's [Professor George Cybenko](#).

or improved versions of the components in previous images. If we want to cover this broader range of inputs then new images might need to be included in the training data. Stated another way, we should not expect the supervised machine learning system to perform well on inputs that are very different from the inputs it has seen in its training data. The training data must be *representative* of the inputs that the system must process in the future.

A more subtle and significant assumption in the ability of a neural network to learn from training data concerns *the size of the training data set*. In general we can improve the performance (e.g., the accuracy) of a supervised machine learning system by providing it with more training data.

Nevertheless, there must still be a large enough data set for learning to be achieved with the desired accuracy. That is, the training data must contain a large enough number of input-output pairs to cover the variation that is possible in the inputs. In general, the more complex the task, the larger the training data set must be. Very complex learning objectives may require that the size of the training data set be unreasonably large in order to represent all possible variations. A large enough data set simply may not exist, and may be clearly impossible to create in any reasonable amount of time. For example, creating ten trillion annotated images would take years for hundreds of people to manually create. Many neural network tasks that require higher level language understanding and / or significant domain expertise can fall into this category because very complex relations would underlie higher-level understanding and meaning.

Lack of interpretability

Neural networks and other machine learning models learn a complex mathematical function that transforms the inputs to the outputs that are produced by the network. One prominent drawback of the machine learning approach is that it is usually not apparent what *exactly* the model has learned to do. The system might provide accurate decisions for hundred's of thousands of inputs that are used to test the system's performance. However, that does not tell us how exactly the system is transforming inputs to outputs to make its decisions. Remember, a human has not written rules that tell the software how to perform its task. The neural network has effectively created its own rules from the data without any human guidance on how to perform its task.

Absent any insights into how the network makes its decisions, we cannot guarantee that it will perform well on all inputs it might see in the future. The network's decision-making process is not

explainable or *interpretable*. This can make it hard to trust that the machine learning system will continue making acceptable decisions. The lack of interpretable outputs is a major drawback in using neural network and other machine learning models for critical business decisions. One common problem is the risk the network makes *biased* decisions. Since the neural network's decision process is opaque, there may not be any indication that a biased decision is possible until it is made, or possibly only long after that decision is made.

Some systems take a hybrid approach that combines aspects of both the machine learning and rules-based approaches. In this hybrid approach the machine learning model is created with some knowledge that a human expert has about the decision making process that should be followed, or with some assumptions a human expert has about the properties of data. Although this is not quite the same as a rules-based approach, it does leverage domain knowledge, much like rules leverage domain knowledge to explicitly specify how the software must make its decisions.

Data and machine learning

Supervised machine learning systems require labeled training data, typically in large amounts. Labels essentially overlay human judgment on the raw input to be processed. This overlay is useful precisely because it combines human judgments with raw input, which carries no explicit human judgments. Acquiring labeled data can be an onerous process that requires a great deal of human time and effort. The larger the data set, the more costly it is to label that data set. For example, to create a training set of assembly line images a human would need to review each image and assign it to a class such as “defective” or “not defective”. The human(s) assigning these classes would have to do so with great accuracy; incorrect labels could train the machine learning software to perform its task incorrectly. The need to create large amounts of labeled training data can be an obstacle in designing supervised machine learning software that performs a new task for which no training data exists. This is a very common problem when designing machine learning systems for use in industries which have not yet embraced machine learning.

Labeling existing data

To create a new labeled data set a firm may look to internal or external annotaters. It can be simplest to use employees internal to the firm, at least because one can control the annotation process from start to finish. The firm can specify aspects of the annotation process such as (a) the manner in which people must annotate each type of input, (b) how the quality of annotated data is reviewed and if necessary improved, (c) the skill level of the people assigned to the annotation team, and (d) the time by which annotations must be completed. Typically the firm must pay for annotation by its employees, if not directly then indirectly by the opportunity cost of allocating staff to annotation rather than to other work.

A less obvious way to annotate a data set internally is to capture annotations that either are being created by staff already, or that could be created with straightforward modification of existing workflows. As a simple example of the former, imagine that employees who review PDF copies of sales orders currently copy those orders to a different folder depending on the type of sale: residential customer, business customer, or third-party intermediary. The folder destination can serve as a label for each sales order. This label information is currently being generated each time an employee copies a PDF to a folder. It would be fairly simple for a program to create a data set consisting of each sales order and its corresponding folder.

As another example, imagine that the firm would like to understand which emails are considered most important by a team of equity analysts. There is no existing process by which the analysts designate emails as important, so these kinds of labels are not currently being generated by employees. Nevertheless, it would be relatively simple to capture this information. The analysts' email interface could be altered to include a button that, when pressed, signals that the analysts consider the email to be important. The email system would hereby be capable of collecting the text of each email as well as whether the analyst labeled that email as important, thus creating the labeled data set.

It may be the case that annotation can be done externally more expeditiously than internally. Annotators can be hired to perform labeling work. [Google's Vertex AI](#) platform and [Clarifai](#) each provide data labeling service for a fee. [Amazon's Mturk](#) is a similar service but provides access to a much larger and diverse marketplace of people who will label data for a fee. Each person might only annotate a very small number of examples in the data set, yet the efforts of dozens or hundreds

of annotators collectively can accomplish impressive results with enough time and resources. Generally a service that allows a large group of people to perform a small, quick service such as annotating a handful of examples is called *crowdsourcing*.

Other forms of external data annotation require more strategy to establish, but can be cheaper or faster than simply hiring annotators. Contests and competitions to label data can essentially be a form of crowdsourcing in exchange for the possibility of recognition or accolades rather than pay. Even more interesting is to create opportunities for the public to annotate data without even realizing that they are doing so. [Video games](#) have been designed to obtain data annotation from players. [Captcha](#) and other tools that protect websites against automated access also collect annotations from users. The user is asked to prove they are human by answering a question or selecting an image that meets certain criteria. The captcha task is by design something that software cannot do well, so it can provide valuable human judgment that can serve as data labels.

External annotation can have shortcomings. Some data cannot be annotated by people external to the firm. For example, confidential, valuable, or sensitive data often cannot be released to the public or to any external annotators. Even if safeguards such as confidentiality agreements are employed there is always the possibility that they could be violated. Determining which of many annotators violated such an agreement, or that a violation has occurred, might not be apparent. More generally, the limitations of external data annotation generally mirror the advantages of internal annotation. Depending on the annotators, it may be difficult to control the quality or speed of annotation.

Rather than crowdsourcing to label existing data, training data with labels can be created entirely by crowdsourcing. For example, the online game [Quick, Draw!](#) asks players to draw specified objects, such as “cell phone” or “duck”. In doing so players contribute examples to a data set of millions of drawings. The input is the drawing and the label is the specified name of the object the player was told to draw. A machine learning system uses this training data to learn to correctly identify drawings by future players.

Creating labeled datasets

Some labeled data sets can be created without a human explicitly labeling data. One method is to use software to transform data into a different form. The data before and after transformation can serve as the training data’s inputs and labels, or vice versa. For example, it is simple to take an

image and reduce its resolution, say from 300 pixels per inch to 100 pixels per inch. If we were to reduce the resolution of thousands of images, we could consider this to be a data set of low-resolution inputs and high-resolution outputs. A supervised machine learning system could be trained to generate high-resolution versions of low-resolution images. Here the original images serve as the labels and the transformed images are the inputs.

A similar transformation can be used to create blurry images from clear images. The resulting data set of blurry inputs and clear labels could be used in training a machine learning system how to sharpen blurry images. Likewise, transforming color images to gray scale delivers a data set to train a system to colorize old black-and-white photos.

Note that in each of the above examples data was easy to transform in one direction, say from color to gray scale or from clear image to blurry. However transformation in the reverse direction is not easy, but is potentially valuable to perform. When some data meets this pattern it is a candidate for automatically creating a labeled data set using software. The transformed data becomes the inputs and the original data is the labels for the inputs. Using software to create useful labeled data can require creativity and insight, but yield data sets too cumbersome to be created manually.

Carefully design the objective machine learning training

Sometimes a bit of creativity is helpful in either identifying a good source of training data that already exists, or creating a labeled data set if one does not already exist. We may also have to think a bit about exactly what task we want the neural network to learn. The task dictates the training objective and therefore the training data that will be required. A slightly different but acceptable task may require a simpler or easier-to-acquire data set.

For example, assume we want to train a neural network to correctly answer questions posed in natural language. A set of questions and their answers, such as in [Yahoo Answers](#), can be used as an existing labeled data set. The questions would be the inputs and the correct answers would be the desired outputs, i.e., labels. However, an alternative training objective is to *select* the best answer from a set of several candidate answers. For this task, the training data consists of inputs (questions and candidate answers) and outputs (relevance scores for each candidate answer). Although these two training objectives are similar in that both attempt to answer natural language questions, one is much harder than the other. The second task is somewhat analogous to answering a multiple choice

question while the first task is analogous to answering the more difficult “fill in the blank” question. Scoring a candidate answer requires that the neural network merely predict a single numerical score, while the other task requires that the neural network generate a sequence of text that forms a coherent and accurate answer to the question.

Self Assessment Questions

You should spend enough time perusing this section to be able to answer the following questions.

What generally is a neural network and what does it do?

Which of the four types of machine learning is involved in neural network?

What increases the power of neural networks? In this context what do we mean by increased power?

How do you arrange a neural network to handle classification tasks.

What do we mean about the interpretability of neural network? What are the implications of this?

What are some commonly-referenced statements about neural network training that are true but misleading?

What are some considerations in labeling an existing unlabeled data set?

What kinds of data labeling should not be crowdsourced?

How exactly does the Quick, Draw! Game generate a labeled data set? What are the inputs and labels ?